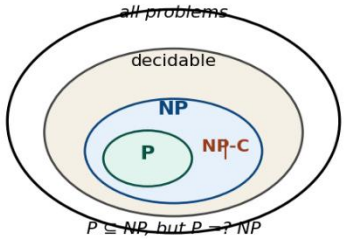
Tarjan	SCC algorithm (depth first search) O(V+E)
Dijkstra	Shortest path (greedy)
Karatsuba	Fast mult (D&C)
Kruskal/Prim	MST (greedy)
S. Cook	SAT is NP-complete
R. Karp	21 NP-complete problems
Rivest/Shamir/Adleman	RSA cryptosystem

Greedy	Dijkstra, Kruskal, Prim, Huffman
Divide & Conq	Karatsuba, mergesort, FFT, closest pair
Dynamic Prog	LCS, Floyd-Warshall, Bellman-Ford, knapsack, Fib

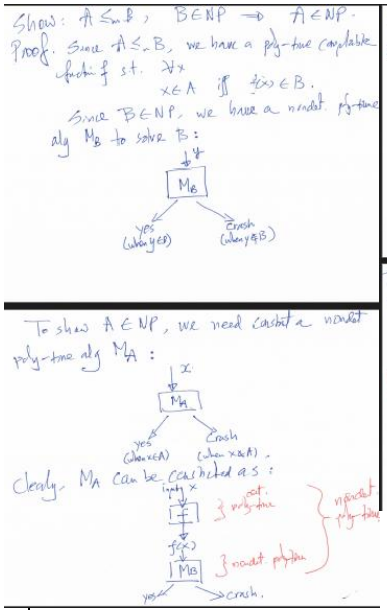
P, NP, NP-Complete

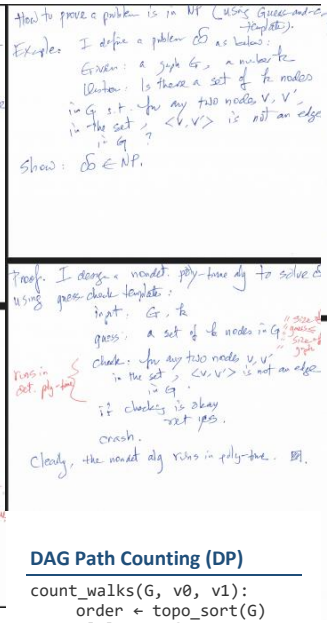
- **P:** class of problems solvable by *deterministic* algorithms in poly time.

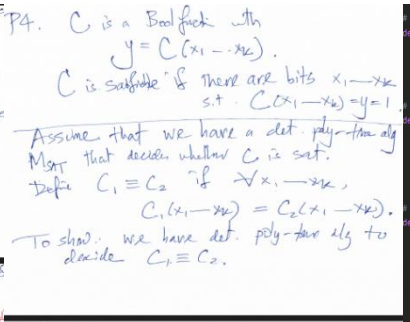


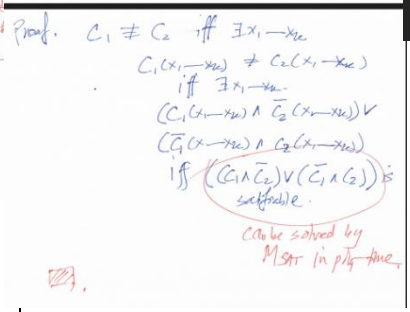
- **NP:** solvable by *nondeterministic* algorithms in poly time.
- **P ⊆ NP**. Open: P =? NP.
- **A ≤$_m$ B:** ∃ poly-time computable f s.t. ∀x, x ∈ A ⟺ f(x) ∈ B.
- **A is NP-complete:** (1) A ∈ NP, (2) ∀ B ∈ NP, B ≤$_m$ A.
- **A is NP-hard:** condition (2) only.

Four Reduction










```
# RR(x1,x2,y1,y2) given, Compute R²:
def R2(RR):
    RR_xz = RR.compose({y1: z1, y2: z2})
    RR_zy = RR.compose({x1: z1, x2: z2})
    return (RR_xz & RR_zy).smoothing([z1,z2])

# R⁴: same idea, composing R² with itself.
def R4(RR):
    R2_val = R2(RR)
    R2_xz = R2_val.compose({y1: z1, y2: z2})
    R2_zy = R2_val.compose({x1: z1, x2: z2})
    return (R2_xz & R2_zy).smoothing([z1,z2])

# R*: iterate until fixed point.
def Rstar(RR):
    Rstar = (x1.iff(y1)) & (x2.iff(y2)) | RR
    prev = None
    while Rstar != prev:
        prev = Rstar
        Rstar_xz = Rstar.compose({y1: z1, y2: z2})
        Rstar_zy = Rstar.compose({x1: z1, x2: z2})
        Rstar = Rstar | (Rstar_xz & Rstar_zy).smoothing([z1,z2])
    return Rstar
```

Lemmas

```
1.  A ≤m B,  B ≤m C   ⟹   A ≤m C   (transitivity)
2.  A ≤m B,  B ∈ P    ⟹   A ∈ P    (P-closure)
3.  A ≤m B,  B ∈ NP   ⟹   A ∈ NP   (NP-closure)
4.  A ≤m B            ⟹   Aᶜ ≤m Bᶜ (complement)
```

Guess/Check Template (prove A ∈ NP)

```
input:  x
guess:  certificate y with |y| ≤ poly(|x|)
check:  predicate(x, y) in det. poly time
        if check ok: return YES
        else: crash
```

Key: *guess size ≤ input size. Check must be deterministic poly time.*

Encoding Size Trap (Unary vs Digits)

UNARY n: "111...1" → input size = n. So O(n²) trial division is poly. Unary FACT ∈ P (no guess needed).

DIGIT n: input size = $\log_{10}$ n. O(n²) is exponential. FACT (digits) only known to be in NP (guess p,q,r ≤ n).

NP-Complete Problem Lineup

Known NP-complete: SAT, 3SAT, Independent Set (IDSET), Vertex Cover (VC), Hamiltonian Cycle (HC), TSP.

Status unknown: Graph Isomorphism (GI), FACT (digit form).

Outside NP (decidable but harder): EXPSPACE-complete, undecidable problems (halting).

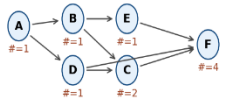
States: (q_1, q_2). **Trans:** δ((q_1,q_2),a) = ($\delta_1(q_1,a)$, $\delta_2(q_2,a)$). Accept iff both. Recognizes $L_1 \cap L_2$.

LCS with Pattern Recipe

```
Mα × Mβ × M_pattern = M (finite)
M is DAG → run longest path
```



States: pairs (q_1,q_2). Accept iff both accept. Transitions componentwise.



Topo: A,B,D,E,C,F — forward propagation gives #walks A→F = 4

DAG Path Counting (DP)

```
count_walks(G, v0, v1):
    order ← topo_sort(G)
    N[v] ← 1 if v=v0 else 0
    for u in order:
      for u→v:  N[v] += N[u]
    return N[v1]   # O(V+E)
```

DAG Path Counting Variants

Even-length walks: state = (node, parity). E[v] += O[u], O[v] += E[u] for each edge u→v. Return E[v_1].

More reds than greens (HW): state = (node, red_count, green_count). Update tuple along each colored edge. Sum entries with red > green.

SCC Application Cheat Table

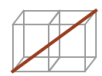
Connected? all of G forms ONE SCC.

Is DAG? every SCC is a single node (no self-loops).

ω-walk hits v_2 ∞-often? reachable looping SCC contains v_2.

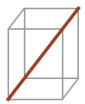
Walk covers all nodes (HW#3)? SCC condensation is a "line".

Case 1: D ≤ √6



√(1²+2²+1²) = √6

Case 2: D ≤ √3

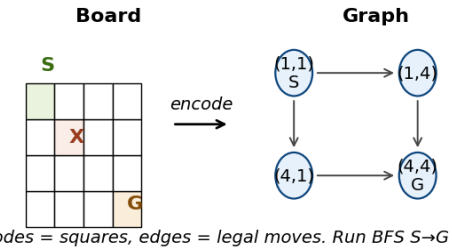


√(1²+1²+1²) = √3

D ≤ max{√6, √3} ≤ 3 → search radius = 3

- n^3 planes in n×n×n cube, dist ≥ 1.
- Pigeonhole: each unit ≤1 plane (D ≤ √6) OR some unit ≥2 (D ≤ √3).
- D ≤ 3 → search radius 3, bounded candidates per plane.
- Bucket trick: plane (1.32, 4.7, 2.88) → unit (1, 4, 2). O(n^3).

BDD Project (PyEDA)



Nodes = squares, edges = legal moves. Run BFS S→G in O(k²).

Nodes: non-forbidden squares. Edges: legal moves. BFS for reachability. O(k²).

Local Propagation (DFA walk count)

Count walks of length L in a DFA: each node i has count N_i. Update all in parallel: $N'_j = \Sigma_i N_i$ over edges i→j. Repeat L times.

```
# Example: walks of len 10 in DFA
N = [1, 0, 0, 0]   # start at node 0
for step in range(10):
  N' = [0]*4
  for each edge i→j: N'[j] += N[i]
  N = N'
return sum(N[f] for f in finals)
```

Min-Product Path

Log trick: log(∏w_i) = Σ log(w_i). Run Dijkstra on log weights (≥ 0 since w_i ≥ 1). Or modify Dijkstra: × instead of +, init dist[src]=1.

```python
def fibonacci_linear(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

ω-Walk Examples (SCC Apps)

Q: ω-walk from v_1 hits v_2 infinitely, hits v_3 only finitely?

Drop v_3 from G. Run SCC.
Find looping SCC reachable
from v_1 that contains v_2.
Return YES iff such SCC exists.

Q: ω-walk hits v_2 ∞-often AND v_3 ∞-often?

Find looping SCC reachable from v_1
that contains BOTH v_2 and v_3.

FACT in NP (digit form)

Given n in digits (size = log n), show: ∃p,q,r ≥ 2 with n = pqr is in NP.

```
input   n (in digits)
guess   p, q, r each ≤ n
        (each is poly bits)
check   p,q,r ≥ 2 AND p·q·r = n
        (multiply: poly time)
        if ok: YES, else crash
```

NFA → DFA (subset construction)

States of DFA = subsets of NFA states. Start = {q_0 ∪ ε-closure}. Accept if any subset element is final. Worst case 2^n states.

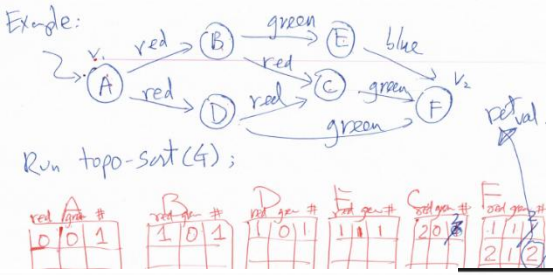
Question

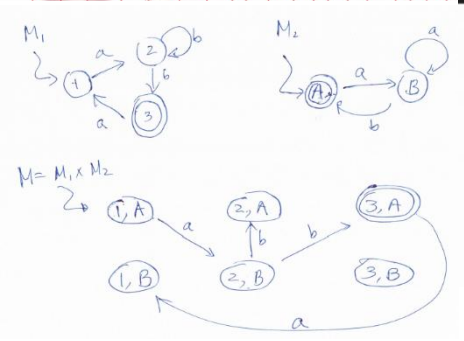
Given a graph G with two designated nodes v_1, v_2: is there an infinite walk from v_1 such that v_2 is passed infinitely many times and v_1 is passed finitely many times?

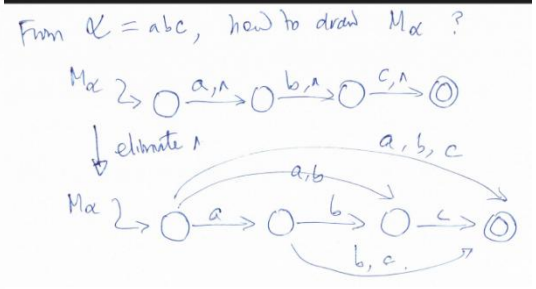
Algorithm

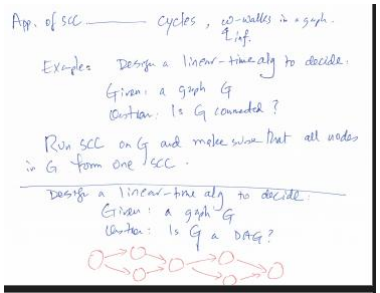
1. Build $G' = G \setminus \{v_1\}$ (delete v_1 and its incident edges).
2. Compute the SCCs of G'.
3. Let S be the SCC containing v_2. Check that S is **nontrivial**: either $|S| \geq 2$, or v_2 has a self-loop in G'. If not, return **No**.
4. Run BFS/DFS from v_1 in G. If any vertex of S is reached, return **Yes**; otherwise return **No**.

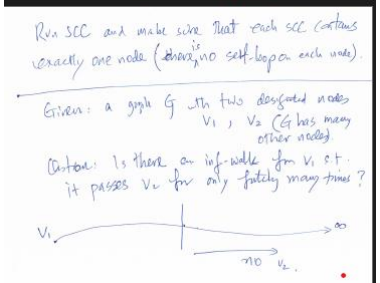
Runtime: $O(|V| + |E|)$.

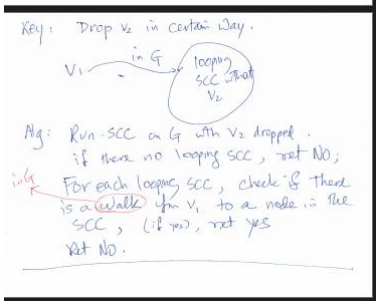













```
Given: n³ airplanes in n×n×n cube, pairwise dist ⩾ 1.
Goal: closest pair in linear (= O(n³)) time.

REASONING:
1. Partition the n×n×n cube into n³ unit cubes (1×1×1 each).
2. By pigeonhole, since we have n³ planes and n³ units, either:
    Case 1: every unit holds exactly one plane, OR
    Case 2: some unit holds at least two planes.
3. Let D = closest-pair distance. Bound D in each case:
    Case 1: closest pair lies in adjacent units. Max distance
            across two adjacent units = √(1² + 2² + 1²) = √6.
            So D ⩽ √6.
    Case 2: two planes share a unit. Max distance within a unit
            = unit diagonal = √(1² + 1² + 1²) = √3.
            So D ⩽ √3.
4. Therefore D ⩽ max{√6, √3} = √6 ⩽ 3.

ALGORITHM (using bucket trick):
1. Create a 3D bucket array B[n][n][n], initially empty.
2. For each plane p at (x, y, z):
    map p to bucket B[⌊x⌋][⌊y⌋][⌊z⌋].
3. For each plane p, scan buckets within radius 3 of p's bucket
   (at most 7³ = 343 buckets, each holding O(1) planes since
   pairwise distance ⩾ 1 means each unit has bounded planes).
   Compute actual distance to every candidate; track the minimum.
4. Return the closest pair found.

TIME ANALYSIS:
- Step 2: O(n³) work to bucket all planes.
- Step 3: O(343) per plane × n³ planes = O(n³).
- Total: O(n³) = O(input size). LINEAR TIME. ✓
```

P4. (likely in final). However, we only need solve the following:

(*): For the G, for given two nodes u, v, whether there is a walk u → v on which #red ≡₅ 0. (you call UR SCC to solve it.)

From G, u, v, I construct a FA M to simulate a walk from u → v while using memory to keep track of the value (#red mod 5). M says yes iff at the end the value is zero.

Note that (*) says yes iff L(M) ≠ ∅. we do have linear-time alg to solve it (3/7).

RSA alg. (1978) // all msgs - all keys are numbers

1. Pick two very large primes, p, q. Compute n = p·q.
2. Pick a small odd number e s.t. gcd(e, (p-1)(q-1)) = 1. // e and (p-1)(q-1) are co-prime
3. Compute d s.t. ed mod (p-1)(q-1) = 1.
4. public key: (e, n) // destroy p, q.
   private key: (d, n). //

Encryption: C = (M^e mod n).
Decryption: M = (C^d mod n)

Worries: ① how many prime #s? density. Euler's prime density ~ n/ln n. # of primes with the range 1 ~ n.

② security of RSA relies on: there is no efficient (poly-time) alg

FACT: Given: n. Want: two primes p, q s.t. n = p·q. — large # factorization.

ob1: ② of ②: the security of RSA relies on an assumption (not a fact). It's an open problem whether RSA is secure.

// FACT ∈ NP but we don't know whether it's NP-complete. (next wk).

// Recall: time complexity is measured (is a function) on input size (NOT input value.)

ob2: I have the following brute-force alg to solve FACT:
input n
Enumerate all p, q ⩽ n s.t. check n = p·q and p, q are primes.
primetesting can be solved in poly-time. (IIT Kanpur)
If yes → ret p, q

Time complexity of my brute-force alg:
① n^2 iterations on $1 \leq p, q \leq n$.
② Assume that primetesting can be done in Constant time;
③ total runtime: $O(n^2) = O(2^N)$
size of input n: $N = \log n$
p exp!